



EMI305 · PRODUCCIÓN DE SOFTWARE · SISTEMAS CIBERFÍSICOS

Análisis de un lenguaje de modelado visual: el DSL PIM para CPS (AO-MDD4CPS)

Constructos, relaciones, restricciones y mecanismos de representación de un lenguaje de dominio específico para sistemas ciberfísicos.

Magíster en Ingeniería Informática · UFRO

EMI305 · Producción de Software · 2026



INSTITUCIÓN ACREDITADA
6 AÑOS
EN TODAS LAS ÁREAS
• GESTIÓN INSTITUCIONAL • DOCENCIA DE PREGRADO
• DOCENCIA DE POSTGRADO • INVESTIGACIÓN
• VINCULACIÓN CON EL MEDIO

¿Qué es y para qué sirve el DSL PIM para CPS?

Propósito. Describir la **arquitectura** de un sistema ciberfísico a nivel **PIM** (independiente de plataforma), dentro del proceso **MDD4CPS**. Es el eslabón entre el modelo de objetivos (*CIM / iStar 2.0*) y el código (*PSM / C++*).

Qué permite representar. Componentes ciberfísicos con comportamiento **periódico** (percepción y actuación), **acciones bajo demanda**, **recursos** de hardware y software, y **mensajería** entre nodos físicos.

Contexto de uso. Transformación CIM → PIM en la herramienta [aomdd4cps](#) (Navarro et al., 2025), editando el diagrama en **diagrams.net** y serializándolo como XML (mxGraph). En este análisis se ilustra con **SVIF** — videovigilancia con identificación facial, dos nodos ESP32.

POR QUÉ UN DSL Y NO SOLO UML

Conceptos de primera clase

El dominio CPS incorpora nociones que en UML exigirían perfiles o extensiones. El DSL las expresa como constructos propios:

HILOS PERIÓDICOS

RECURSOS DE HARDWARE

MENSAJES ENTRE NODOS

«Identificar un rostro cada **500 ms** y publicar el evento»

— expresable sin comprometerse aún con una plataforma.

Constructos, relaciones y reglas de buena formación

CONSTRUCTO	TIPO (XML)	ROL
CPCComponent	cps_component	Nodo ciberfísico (contenedor)
OperationalGoal	operational_goal	Objetivo periódico «OnInterval»
Action	action	Tarea bajo demanda «OnDemand»
RefOperator	and/or_ref_operator	Refinamiento AND / OR
SWResource	sw_resource	Dato persistente (estructura)
HWResource	hw_resource	Recurso físico (pin / periférico)
CommThread	comm_thread	Emisor de mensajes
ListenerThread	listener_thread	Receptor de mensajes

RELACIONES

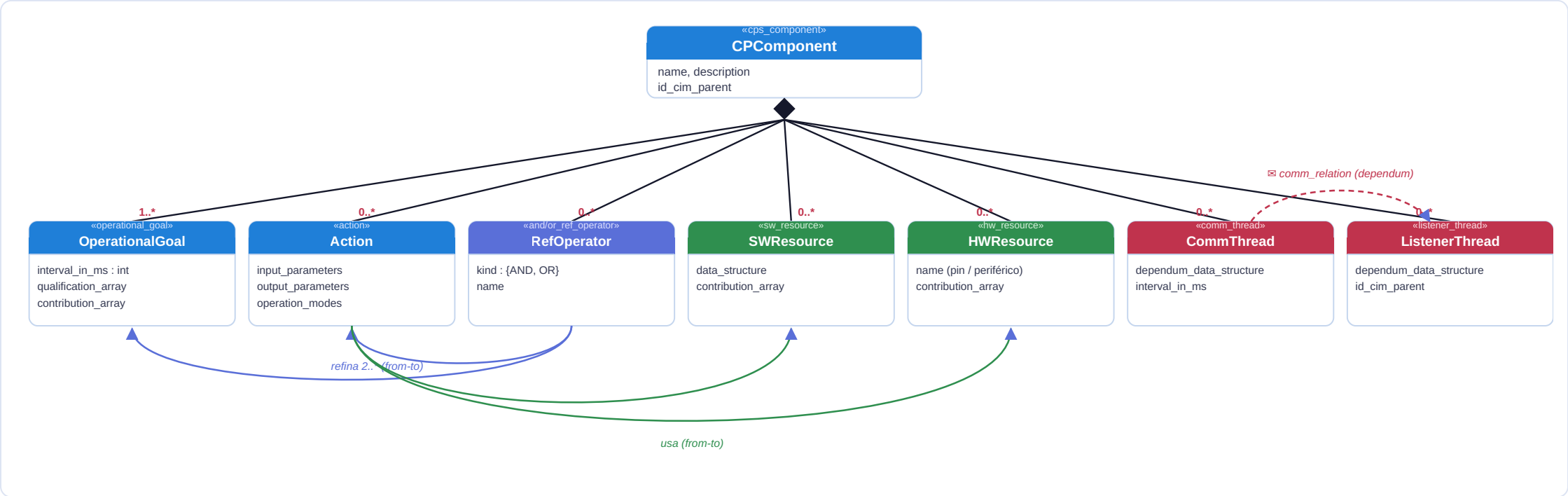
relation_from_to — arista dirigida de **contención**, **refinamiento** (operador → hijos) y **uso** (acción → recurso).

comm_relation — canal de mensaje **entre** componentes; transporta el *dependum* y se decora con el símbolo de sobre **mail_symbol** ✉.

Restricciones (buena formación):

- Todo constructo vive dentro de un **CPCComponent**.
- Un **OperationalGoal** debe declarar **interval_in_milliseconds** > 0.
- Un **RefOperator** agrupa ≥ 2 hijos.
- Un **comm_relation** conecta **exactamente** un CommThread (emisor) con un ListenerThread (receptor); el *dependum* se define en el emisor.
- Cada constructo conserva **id_cim_parent** (trazabilidad al CIM).

Metamodelo simplificado (diagrama de clases)



◆ composición (contención en CComponent) · → *relation_from_to* · → *comm_relation*. Los qualification/contribution arrays enlazan cada elemento con los softgoals del CIM.

La notación: mecanismos de representación visual



CPCComponent
Swimlane: contenedor del nodo



Action
Rectángulo recto (bajo demanda)



SWResource
Cilindro (dato / estructura)



Comm / Listener
Trapezio (emisor / receptor)



OperationalGoal
Rect. redondeado (periódico)



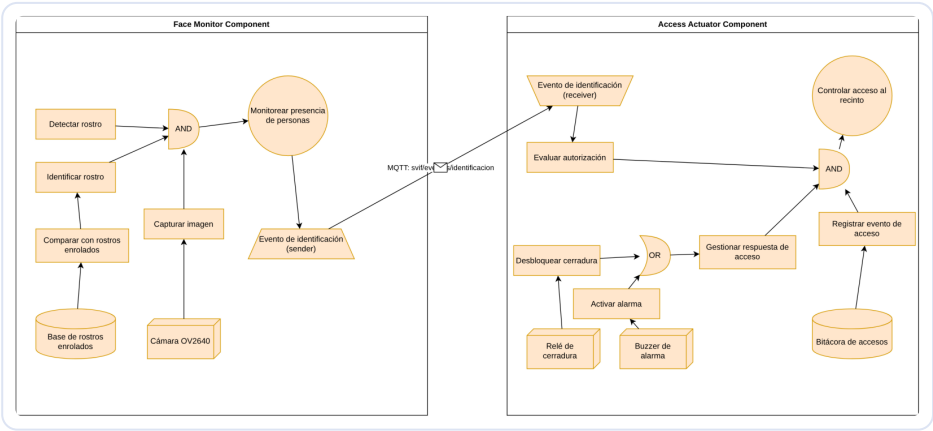
RefOperator
Compuerta AND / OR



HWResource
Cubo (recurso físico)



comm_relation
Canal de mensaje + sobre



Modelo PIM real de SVIF construido con esta notación (dos swimlanes CPCComponent, acciones en naranja, recursos, y el canal MQTT: svif entre nodos). Fuente: [pim-dsl-svif.drawio](#).

El color naranja es común a los elementos de comportamiento y recurso; la **forma** es lo que distingue el constructo. Muchos atributos (períodos, parámetros) viven **dentro** del objeto, no en el lienzo.

Significado: cómo se interpreta cada constructo

La semántica del DSL está **definida por traducción** hacia un lenguaje destino (C++ / Arduino con FreeRTOS): cada constructo tiene un significado operacional preciso que la transformación PIM → PSM materializa.

CONSTRUCTO	SIGNIFICADO (INTERPRETACIÓN)	REALIZACIÓN EN EL PSM
CPCComponent	Unidad de despliegue autónoma (un nodo físico)	Sketch <code>.ino</code> + <code>comm_utils.h</code> + <code>secrets.h</code>
OperationalGoal	Comportamiento que se ejecuta cada <i>interval_in_ms</i>	Hilo periódico FreeRTOS con ese período
Action	Operación invocable bajo demanda con entradas/salidas	Función C++ con firma derivada de los parámetros
AND / OR operator	AND: todos los hijos se ejecutan · OR: se elige una rama	<i>secuencia</i> / condicional <code>if / else</code>
SWResource	Dato persistente estructurado del componente	<code>struct</code> tipada (p. ej. <code>EnrolledFace</code>)
HWResource	Punto de integración con el mundo físico	Comentario + pin (<code>RELAY_PIN</code> , <code>BUZZER_PIN</code>)
Comm / Listener	Publicación / recepción del <i>dependum</i> entre nodos	Hilos MQTT (publish / callback) con la <code>struct</code> del <i>dependum</i>

Lectura de un modelo: «este nodo **percibe** cada 500 ms, ejecuta estas acciones sobre estos recursos, y **publica** un evento que el otro nodo **recibe** y usa para **actuar**» — todo **sin decir** aún en qué placa ni con qué protocolo.

¿Es adecuado el lenguaje? Ventajas, límites y mejoras

¿Adecuado para el dominio? Sí para CPS pequeños–medianos: expresa periodicidad, recursos y mensajería como conceptos de primera clase y **posterga la plataforma** al nivel correcto. Su valor crece con el número de componentes y dependencias, donde el andamiaje de comunicación es lo más propenso a error si se escribe a mano.

VENTAJAS OBSERVADAS

- Trazabilidad `id_cim_parent`: cada elemento enlaza con su objetivo/tarea del CIM.
- Andamiaje **homogéneo** de hilos y comunicación → menos errores de concurrencia.
- Decisiones **postergadas** al nivel adecuado (plataforma sólo en PSM).
- **Serializable en XML** → transformable con XSLT / scripts.

LIMITACIONES (VERIFICADAS EN LA SEMANA 4)

- No expresa **restricciones temporales duras**: el período es un atributo, pero nada verifica su cumplimiento.
- Los **softgoals** sobreviven sólo como **comentarios** (no verificables automáticamente).
- La herramienta `aomdd4cps` descarta AND/OR en la transformación CIM → PIM.
- Notación **cargada de atributos ocultos**: hay que abrir cada objeto para ver período o parámetros.

Qué mejoraría: constructos para **plazos y prioridades** de hilos; un **validador** de buena formación; hacer **visibles** en la notación los atributos clave (período); y un mecanismo para **verificar softgoals**, no sólo comentarlos.

Un lenguaje bien definido: los cinco elementos

Lo que caracteriza al DSL PIM para CPS

- **Propósito claro:** arquitectura CPS independiente de plataforma.
- **Sintaxis abstracta:** 8 constructos con relaciones y restricciones.
- **Metamodelo:** composición en CPComponent + refinamiento + uso + comunicación.
- **Sintaxis concreta:** la **forma** distingue el constructo (swimlane, rect, cilindro, cubo, compuerta, trapecio, sobre).
- **Semántica:** definida por traducción a hilos FreeRTOS, funciones y structs.

Un DSL **bien definido** = las tres capas (abstracta + concreta + semántica) coherentes entre sí.

Referencias:

Fowler, M., & Parsons, R. (2010). *Domain-Specific Languages* (Addison-Wesley Signature Series). Addison-Wesley.

Mernik, M., Heering, J., & Sloane, A. M. (2005). *When and how to develop domain-specific languages*. *ACM Computing Surveys*, 37(4), 316–344.

Aßmann, U., Zschaler, S., & Wagner, G. (2006). *Ontologies, meta-models, and the model-driven paradigm*. Springer.

Navarro, C., Devia, L., Labra Gayo, J. E., & Cares, C. (2025). *An agent-oriented model-driven development process for cyber-physical systems*. *CibSE 2025* (pp. 150–164). SBC.

Dalpiaz, F., Franch, X., & Horkoff, J. (2016). *iStar 2.0 language guide*. *arXiv:1605.07767*.

Repositorio MDD4CPS: github.com/mdd4cps/aomdd4cps (CC BY-NC 4.0).